

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

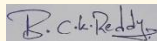
### CO1 ASSESSMENT TOOL 1 – Case Study Analysis

|                |                                                                                                              |               |                                         |
|----------------|--------------------------------------------------------------------------------------------------------------|---------------|-----------------------------------------|
| Course Code:   | CSA10 / CSA1023                                                                                              | Course Title: | Software Engineering                    |
| CO Assessed:   | CO1                                                                                                          | Assessment:   | Assessment Tool 1 – Case Study Analysis |
| Weightage:     | 20%                                                                                                          | Total Marks:  | 25                                      |
| CO1 Statement: | Apply advanced methodologies and agile project management techniques to manage software projects effectively |               |                                         |
| Student Name:  | B. Charan Kumar Reddy                                                                                        | Reg. No.:     | 192472195                               |
| Date:          | 06/10/2026                                                                                                   | Duration:     | 60 minutes                              |

#### Code of Conduct

I B. Charan Kumar Reddy (192472195) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



#### Annexure A – Case Study Analysis

##### Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

##### 1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

##### 2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

##### 3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

##### 4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

##### 5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

**6. Discuss Expected Outcomes and Limitations**

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

## **Answer**

### **Problem Statement: Autonomous Coding Agent Security and Software Supply-Chain Protection Platform**

The proposed platform continuously monitors AI coding agents, third-party dependencies, repositories, and CI/CD workflows to detect suspicious activities such as unauthorized tool execution, privilege escalation, secret exposure, malicious dependency changes, and abnormal repository modifications. It maintains SBOMs and software supply-chain audit trails, enforces dependency security policies, provides risk scoring, supports human approval for sensitive agent actions, and enables automatic isolation or rollback of suspicious changes.

#### **1. Understand the Problem Statement**

##### **a) Problem Understanding**

Modern software development increasingly uses AI coding agents, third-party open-source dependencies, source-code repositories, and automated CI/CD workflows. AI coding agents can generate code, modify files, install dependencies, execute commands, and interact with external tools. These capabilities improve development speed but also create security risks when an agent performs an unauthorized action or when a trusted dependency, repository, or development workflow is compromised. Traditional monitoring may not detect such activity because it can occur through legitimate development tools.

##### **b) Objectives**

The main objectives are to:

- Monitor AI-agent actions, repository changes, dependencies, and build activities.
- Detect suspicious dependency and software supply-chain behavior.
- Identify secret leakage, privilege escalation, and unauthorized actions.
- Maintain dependency inventories, Software Bills of Materials (SBOMs), and audit trails.
- Integrate security checks with CI/CD pipelines.
- Support automated rollback and incident-response mechanisms.
- Provide secure, ethical, and accountable governance of AI coding agents.

**c) Scope**

The scope covers security monitoring and governance across the software development supply chain. It includes AI-agent activities, repository modifications, third-party dependencies, SBOM management, CI/CD workflows, security checks, risk assessment, audit trails, human approval, isolation, and rollback. It focuses on securing the development and delivery process and does not replace the AI coding agent, repository platform, or CI/CD platform.

**2. Key Stakeholders and Challenges**

**a) Stakeholders**

The major stakeholders are developers, security administrators, DevOps/CI-CD engineers, project managers, AI coding agents, system administrators, and organizational management.

**b) Roles and Responsibilities**

Their roles and responsibilities are:

| Stakeholder              | Role and Responsibility                                                                                                              |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Developers               | Develop and modify software, use AI coding agents, manage dependencies, review changes, and respond to security alerts.              |
| Security Administrators  | Define security policies, configure risk thresholds, monitor alerts, investigate incidents, and approve or reject sensitive actions. |
| DevOps / CI-CD Engineers | Manage build, testing, and deployment pipelines and integrate security checks into CI/CD workflows.                                  |
| Project Managers         | Coordinate project progress and resources while ensuring security requirements are considered.                                       |
| AI Coding Agents         | Generate code, modify files, install approved dependencies, and execute authorized development tasks.                                |

|                           |                                                                                                                         |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------|
| System Administrators     | Manage infrastructure, access permissions, system availability, isolation, and recovery operations.                     |
| Organization / Management | Define organizational security expectations, evaluate software risks, and support security and compliance requirements. |

### c) Challenges

The major challenges include:

- Unauthorized or unintended actions by AI coding agents.
- Compromised or vulnerable third-party dependencies.
- Accidental exposure of credentials, tokens, or API keys.
- Privilege escalation caused by excessive permissions.
- Malicious build commands or CI/CD pipeline manipulation.
- Fragmented security information across agents, repositories, dependencies, and CI/CD systems.
- Balancing development speed with security, reliability, and accountability.

## 3. Analyze the Current Approach

### a) Existing System

In a typical development process, developers use AI coding tools and source-code repositories while depending on third-party packages. CI/CD pipelines automatically build, test, and deploy software. Security is generally handled through separate controls such as code scanning, dependency vulnerability scanning, secret scanning, repository access control, and CI/CD security checks.

### b) Strengths and Limitations

The strengths and limitations are:

#### Strengths

- Version control maintains the history of source-code changes.

- Dependency scanning can identify known vulnerabilities in third-party packages.
- CI/CD automation provides repeatable build, testing, and deployment processes.
- Access control restricts unauthorized users and operations.

### **Limitations**

- AI-agent actions may not be continuously monitored with sufficient context.
- Security tools may work independently, resulting in fragmented information.
- Suspicious dependency changes may not be identified immediately.
- Individual security alerts may not show the overall software supply-chain risk.
- Sensitive AI-agent operations may occur without explicit human approval.
- Detection may not be directly connected to automatic isolation or rollback.

### **c) Problems and Gaps**

The main gaps are the lack of unified monitoring across AI-agent activities, repositories, dependencies, and CI/CD workflows; limited contextual risk assessment; insufficient human approval for sensitive autonomous actions; and weak integration between detection and automatic isolation or rollback.

## **4. Proposed Software Solution Using Software Engineering Principles**

### **a) Proposed Solution**

The proposed solution is a centralized security platform that continuously monitors AI coding agents, repositories, third-party dependencies, and CI/CD workflows. It collects security-related events, analyzes them using defined policies and risk assessment, and generates alerts when suspicious behavior is detected. Low-risk actions can continue, sensitive actions can require human approval, and high-risk activities can be blocked or isolated. Critical suspicious changes can be rolled back and recorded for investigation.

### **b) Functional Modules**

The major functional areas are:

- AI-agent activity monitoring.

- Repository and change monitoring.
- Dependency inventory and SBOM management.
- CI/CD security checking.
- Risk assessment and alert generation.
- Human approval and governance.
- Isolation, rollback, and incident response.
- Audit trail and security reporting.

### **c) Functional and Non-Functional Requirements**

The functional and non-functional requirements are:

#### **Functional Requirements**

- Monitor AI-agent actions and tool executions.
- Monitor repository modifications and dependency changes.
- Maintain dependency inventories and SBOMs.
- Detect secret exposure, privilege escalation, and unauthorized actions.
- Monitor CI/CD build activities.
- Calculate risk scores and generate alerts.
- Request human approval for sensitive actions.
- Block or isolate suspicious activities.
- Support rollback of suspicious changes.
- Maintain audit trails and security reports.

#### **Non-Functional Requirements**

- Security: protect source code, credentials, policies, and audit information.
- Reliability: provide dependable monitoring and security responses.

- **Scalability:** support increasing numbers of users, repositories, dependencies, and agents.
- **Performance:** minimize the effect of monitoring on normal development activities.
- **Maintainability:** allow individual components to be updated without major changes to the whole system.
- **Usability:** present alerts, risk levels, and approval requests clearly.
- **Auditability:** maintain traceable records of security activities and decisions.

#### **d) Software Engineering Principles**

The software engineering principles are incorporated as follows:

**Modularity:** Security functions are separated into manageable components so each part can be developed and maintained independently.

**Scalability:** The solution can support additional projects, repositories, dependencies, users, and AI agents as usage grows.

**Maintainability:** Clear separation of responsibilities makes it easier to modify individual security functions and add new security rules.

**Reliability:** Continuous monitoring, isolation, and rollback reduce the effect of suspicious or malicious changes.

**Usability:** Clear alerts, risk levels, and approval requests help developers and administrators understand and respond to security events.

**Security:** Access control, least-privilege principles, secret detection, dependency policies, and audit logging protect sensitive development activities.

### **5. Justification of the Chosen Methodology / Framework**

#### **a) Selected Methodology**

Agile Development with DevSecOps Practices is selected for the proposed system.

#### **b) Methodology Justification**

This methodology is suitable because the platform must respond to changing AI capabilities, newly discovered vulnerabilities, evolving security policies, and new software supply-chain threats. Agile supports incremental development and frequent feedback, while DevSecOps integrates security into coding, testing, building, deployment, and monitoring.

### **c) Development and Maintenance**

It supports successful development and maintenance by:

- Supporting incremental development and frequent stakeholder feedback.
- Allowing new security requirements to be incorporated as threats evolve.
- Integrating security testing into development and CI/CD processes.
- Identifying defects and vulnerabilities early.
- Supporting continuous monitoring and rapid response.
- Making continuous maintenance and improvement easier.

## **6. Expected Outcomes and Limitations**

### **a) Expected Benefits**

The expected benefits are:

- Reduced risk of software supply-chain attacks.
- Improved visibility into AI-agent, repository, dependency, and build activities.
- Earlier detection of suspicious or unauthorized changes.
- Better protection against secret exposure and privilege escalation.
- Stronger CI/CD security and software reliability.

- Dependency inventories, SBOMs, and audit trails for better traceability.
- Faster incident detection, isolation, and recovery.
- Improved confidence and accountability in AI-assisted software development.

## **b) Addressing the Challenges**

The identified challenges are addressed through continuous AI-agent monitoring, dependency and SBOM management, secret detection, permission monitoring, CI/CD security checks, repository monitoring, risk-based blocking, human approval, centralized audit trails, isolation, and rollback.

## **c) Risks and Limitations**

Potential risks and limitations include:

- False positive alerts may interrupt legitimate development activities.
- Continuous monitoring may require additional computing resources.
- Integration with different repositories, package managers, and CI/CD platforms can be complex.
- Previously unknown attacks may not always be correctly detected.
- Incorrect detection may cause legitimate changes to be blocked or rolled back.
- The security platform itself must be strongly protected because it handles sensitive development information.

## **d) Future Enhancements**

Possible future enhancements include:

- AI-based anomaly detection for previously unknown behaviors.
- Automated remediation of vulnerable dependencies.
- Integration with external threat-intelligence sources.
- Predictive software supply-chain risk analysis.
- Advanced behavioral analysis of AI coding agents.

- Support for additional repository and CI/CD platforms.
- Automated security-policy generation.

## **Conclusion**

The proposed platform provides a unified approach to securing AI-assisted software development by combining continuous monitoring, dependency protection, risk assessment, human approval, audit trails, isolation, and rollback. Agile with DevSecOps practices provides a suitable approach because the platform must continuously adapt to changing AI capabilities and software supply-chain threats.

## Rubrics for Calculation

| Criteria                                                           | Excellent (5 Marks)                                                                                                                                        | Good (3 Marks)                                                              | Satisfactory (2 Marks)                                                      | Needs Improvement (1 Mark)                                                                        |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Problem Understanding (30%)                                        | Clearly explains the problem, objectives, and scope with complete understanding.                                                                           | Explains the problem with minor omissions.                                  | Basic understanding with limited explanation.                               | Poor understanding; objectives and scope are unclear or missing.                                  |
| Stakeholder & Challenge Analysis (25%)                             | Identifies all stakeholders, their roles, and challenges with clear analysis.                                                                              | Identifies most stakeholders and challenges with adequate analysis.         | Identifies only a few stakeholders or challenges.                           | Stakeholders and system analysis are incomplete or inaccurate.                                    |
| Current System Analysis (20%)                                      | Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.                                                                   | Good analysis with most strengths and weaknesses identified.                | Limited analysis with partial identification of issues.                     | Proposed solution is unclear or inappropriate; requirements are largely missing.                  |
| Proposed Software Solution & Software Engineering Principles (15%) | Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.). | Suitable solution with good application of software engineering principles. | Basic solution with limited application of software engineering principles. | Methodology is inappropriate, poorly justified, or not discussed.                                 |
| Methodology/ Framework Justification (10%)                         | Selects the most suitable SDLC/model/framework and provides strong technical justification.                                                                | Suitable methodology selected with adequate justification.                  | Methodology selected but justification is weak.                             | Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity. |

| Criteria                                                           | Mark       |
|--------------------------------------------------------------------|------------|
| Problem Understanding (30%)                                        | /5         |
| Stakeholder & Challenge Analysis (25%)                             | /5         |
| Current System Analysis (20%)                                      | /5         |
| Proposed Software Solution & Software Engineering Principles (15%) | /5         |
| Methodology/ Framework Justification (10%)                         | /5         |
| <b>TOTAL</b>                                                       | <b>/25</b> |